The following listing shows a sample implementation of the thread-management parts of such a system.

Listing 9.13. Monitoring the filesystem in the background

```
std::mutex config_mutex;
std::vector<interruptible_thread> background_threads;

void background_thread(int disk_id)
{
    while(true)
    {
        interruption_point();                          <-1
        fs_change fsc=get_fs_changes(disk_id);         <-2
        if(fsc.has_changes())
        {
            update_index(fsc);                         <-3
        }
    }
}

void start_background_processing()
{
    background_threads.push_back(
        interruptible_thread(background_thread,disk_1));
    background_threads.push_back(
        interruptible_thread(background_thread,disk_2));
}

int main()
{
    start_background_processing();                     <-4
    process_gui_until_exit();                          <-5
    std::unique_lock<std::mutex> lk(config_mutex);
    for(unsigned i=0;i<background_threads.size();++i)
    {
        background_threads[i].interrupt();             <-6
    }
    for(unsigned i=0;i<background_threads.size();++i)
    {
        background_threads[i].join();                  <-7
    }
}
```

At startup, the background threads are launched ❹. The main thread then proceeds with handling the GUI ❺. When the user has requested that the